```
const questionBank = {
  HTML: [
    {
      q: "Which tags are used to create a link?",
      options: ["<a>", "<link>", "<href>", "<url>"],
      correct: [0]
    },
    {
      q: "Which tags are used to create a table?",
      options: ["<table>", "<tr>", "<td>", "<video>"],
      correct: [0, 1, 2]
    },
    {
      q: "Which tags create lists?",
      options: ["<ul>", "<ol>", "<li>", "<listitem>"],
      correct: [0, 1, 2]
    },
    {
      q: "Which attributes can be used with an image?",
      options: ["src", "alt", "href", "width"],
      correct: [0, 1, 3]
    },
    {
      q: "Which tags are semantic HTML elements?",
      options: ["<header>", "<footer>", "<article>", "<blink>"],
      correct: [0, 1, 2]
    }
  ],

  CSS: [
    {
      q: "Which property changes text color?",
      options: ["color", "font-size", "background", "margin"],
      correct: [0]
    },
```

```
  {
    q: "Which properties can add space outside an element?",
    options: ["margin", "padding", "border", "outline"],
    correct: [0]
  },
  {
    q: "Which properties can control text size or weight?",
    options: ["font-size", "font-weight", "text-size", "font-style"],
    correct: [0, 1, 3]
  },
  {
    q: "Which values are CSS display types?",
    options: ["flex", "grid", "block", "paint"],
    correct: [0, 1, 2]
  },
  {
    q: "Which selectors are valid CSS selectors?",
    options: [".card", "#main", "p", "@button"],
    correct: [0, 1, 2]
  }
],

JavaScript: [
  {
    q: "Which keyword declares a block-scoped variable?",
    options: ["let", "var", "style", "tag"],
    correct: [0]
  },
  {
    q: "Which values are JavaScript data types?",
    options: ["String", "Number", "Boolean", "HTML"],
    correct: [0, 1, 2]
  },
  {
    q: "Which methods can select elements from the DOM?",
    options: [
```

```
      "getElementById",
      "querySelector",
      "getElementsByClassName",
      "makeElement"
    ],
    correct: [0, 1, 2]
  },
  {
    q: "Which operators can compare values?",
    options: ["===", "!==", ">", "print"],
    correct: [0, 1, 2]
  },
  {
    q: "Which can repeat code?",
    options: ["for", "while", "do...while", "heading"],
    correct: [0, 1, 2]
  }
],

RDBMS: [
  {
    q: "Which command retrieves data?",
    options: ["SELECT", "DELETE", "DROP", "ALTER"],
    correct: [0]
  },
  {
    q: "Which commands can change table data?",
    options: ["INSERT", "UPDATE", "DELETE", "SELECT"],
    correct: [0, 1, 2]
  },
  {
    q: "Which are SQL constraints?",
    options: ["PRIMARY KEY", "UNIQUE", "NOT NULL", "PAINT"],
    correct: [0, 1, 2]
  },
  {
```

```
    q: "Which commands are part of transaction control?",
    options: ["COMMIT", "ROLLBACK", "SAVEPOINT", "SELECT"],
    correct: [0, 1, 2]
   },
   {
    q: "Which are common SQL join types?",
    options: ["INNER JOIN", "LEFT JOIN", "RIGHT JOIN", "COLOR JOIN"],
    correct: [0, 1, 2]
   }
  ],

  GK: [
   {
    q: "What is the capital of India?",
    options: ["New Delhi", "Mumbai", "Chennai", "Kolkata"],
    correct: [0]
   },
   {
    q: "Which are planets in our solar system?",
    options: ["Earth", "Mars", "Jupiter", "Moon"],
    correct: [0, 1, 2]
   },
   {
    q: "Which are input devices?",
    options: ["Keyboard", "Mouse", "Scanner", "Monitor"],
    correct: [0, 1, 2]
   },
   {
    q: "Which are programming languages?",
    options: ["Python", "Java", "C++", "HTML"],
    correct: [0, 1, 2]
   },
   {
    q: "Which units are used to measure digital storage?",
    options: ["Byte", "Kilobyte", "Megabyte", "Kilometer"],
    correct: [0, 1, 2]
```

```
    }
  ]
};

const $ = id => document.getElementById(id);

const category = $("category");
const questionEl = $("question");
const optionsEl = $("options");
const progressEl = $("progress");
const timerEl = $("timer");
const progressBar = $("progressBar");
const resultEl = $("result");

let questions = [];
let current = 0;
let answers = [];
let timeLeft = 600;
let timer = null;
let running = false;

function shuffle(array) {
  const copy = [...array];

  for (let i = copy.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [copy[i], copy[j]] = [copy[j], copy[i]];
  }

  return copy;
}

function startQuiz() {
  clearInterval(timer);

  const selectedCategory = category.value;
```

```
questions = questionBank[selectedCategory].map(item ⇒ {
 const optionOrder = shuffle(
  item.options.map((text, index) ⇒ ({
   text,
   originalIndex: index
  }))
 );

 return {
  q: item.q,
  options: optionOrder.map(option ⇒ option.text),
  correct: optionOrder
   .map((option, newIndex) ⇒
    item.correct.includes(option.originalIndex) ? newIndex : -1
   )
   .filter(index ⇒ index !== -1)
 };
});

questions = shuffle(questions);
answers = questions.map(() ⇒ []);
current = 0;
timeLeft = 600;
running = true;

resultEl.hidden = true;
$("startBtn").textContent = "Restart Quiz";
category.disabled = true;
$("submitBtn").disabled = false;
$("prevBtn").disabled = false;
$("nextBtn").disabled = false;

renderQuestion();
updateTimer();

timer = setInterval(() ⇒ {
```

```
    timeLeft--;
    updateTimer();

    if (timeLeft <= 0) {
      submitQuiz();
    }
  }, 1000);
}

function renderQuestion() {
  const item = questions[current];

  questionEl.textContent = item.q;
  progressEl.textContent =
    `Question ${current + 1}/${questions.length}`;

  progressBar.style.width =
    `${((current + 1) / questions.length) * 100}%`;

  optionsEl.replaceChildren();

  item.options.forEach((option, index) => {
    const label = document.createElement("label");
    label.className = "option";

    const checkbox = document.createElement("input");
    checkbox.type = "checkbox";
    checkbox.checked = answers[current].includes(index);

    checkbox.addEventListener("change", () => {
      if (checkbox.checked) {
        answers[current].push(index);
      } else {
        answers[current] = answers[current].filter(
          answer => answer !== index
        );
```

```
    }
   });

   const text = document.createElement("span");
   text.textContent = option;

   label.append(checkbox, text);
   optionsEl.append(label);
  });

  $("prevBtn").disabled = current === 0;
  $("nextBtn").disabled = current === questions.length - 1;
}

function updateTimer() {
  const minutes = Math.floor(timeLeft / 60);
  const seconds = timeLeft % 60;

  timerEl.textContent =
   `${String(minutes).padStart(2, "0")}:${String(seconds).padStart(2, "0")}`;
}

function moveQuestion(step) {
  if (!running) return;

  const next = current + step;

  if (next >= 0 && next < questions.length) {
   current = next;
   renderQuestion();
  }
}

function sameAnswers(selected, correct) {
  return selected.length === correct.length &&
   selected.every(value ⇒ correct.includes(value));
```

```
}

function submitQuiz() {
  if (!running) return;

  running = false;
  clearInterval(timer);
  category.disabled = false;

  const correctCount = questions.reduce((total, item, index) => {
    return total + (
      sameAnswers(answers[index], item.correct) ? 1 : 0
    );
  }, 0);

  const attempted = answers.filter(answer => answer.length > 0).length;
  const incorrect = attempted - correctCount;
  const percentage = Math.round(
    (correctCount / questions.length) * 100
  );

  $("score").textContent =
    `Score: ${correctCount}/${questions.length} (${percentage}%)`;

  $("summary").textContent =
    `Attempted: ${attempted} | Correct: ${correctCount} | ` +
    `Incorrect: ${incorrect} | Unanswered: ${questions.length - attempted}`;

  showReview();

  const key = `quizBest_${category.value}`;
  const previousBest = Number(localStorage.getItem(key) || 0);
  const best = Math.max(previousBest, percentage);

  localStorage.setItem(key, best);
  updateBestScore();
```

```
  resultEl.hidden = false;
  $("submitBtn").disabled = true;
  $("prevBtn").disabled = true;
  $("nextBtn").disabled = true;

  questionEl.textContent = "Quiz completed!";
  optionsEl.replaceChildren();

  resultEl.scrollIntoView({ behavior: "smooth" });
}

function showReview() {
  const review = $("review");
  review.replaceChildren();

  questions.forEach((item, index) => {
    const isCorrect = sameAnswers(answers[index], item.correct);
    const box = document.createElement("div");

    box.className =
      `review-item ${isCorrect ? "correct" : "incorrect"}`;

    const heading = document.createElement("strong");
    heading.textContent =
      `${index + 1}. ${item.q} — ${isCorrect ? "Correct" : "Incorrect"}`;

    const selectedText = document.createElement("p");
    selectedText.textContent = "Your answers: " +
      (answers[index].length
        ? answers[index].map(i => item.options[i]).join(", ")
        : "Not answered");

    const correctText = document.createElement("p");
    correctText.textContent = "Correct answers: " +
      item.correct.map(i => item.options[i]).join(", ");
```

```
    box.append(heading, selectedText, correctText);
    review.append(box);
  });
}

function updateBestScore() {
  const key = `quizBest_${category.value}`;
  const best = localStorage.getItem(key);

  $("bestScore").textContent =
    best === null ? "No score yet" : `${category.value}: ${best}%`;
}

$("startBtn").addEventListener("click", startQuiz);
$("prevBtn").addEventListener("click", () => moveQuestion(-1));
$("nextBtn").addEventListener("click", () => moveQuestion(1));
$("submitBtn").addEventListener("click", submitQuiz);
$("restartBtn").addEventListener("click", startQuiz);

category.addEventListener("change", updateBestScore);

updateBestScore();
```